

Experiment Book 3

Storage, memory, processes and archives | Bash in GitHub Codespaces

Name: _____ Lab ID: _____ Section: _____

Prepare a campus event handover. Measure storage, inspect a running program, stop a designated lab process, and package files so another person can recover them.

Get ready

Sign in with your own GitHub account. Open this link and create a Codespace on **main**. Later, resume the same Codespace to keep working.

<https://codespaces.new/nalinabrol/terminal-lab-3?quickstart=1>

Wait for setup. Choose **Trust Folder & Continue** if asked. A Bash terminal opens. If needed, use **Lab 3: Focus Bash Terminal** in the Command Palette. At the repository root, run:

```
bash terminal-lab-3/start.sh
```

Enter your assigned lab ID, not your email. Run the exact **cd** command printed by setup. Then run:

```
pwd
cat identity.txt
ls
```

My lab root (the complete path from pwd):

Location	Purpose
practice/ and challenge/	Original evidence. Keep it unchanged.
work/	Your reports, notes, copies and archives.

Start every experiment at your lab root. Type commands one at a time. Predict, run, inspect and explain. Write in this book or a separate notes file. Hints are on page 14; the command reference is on page 15.

Setup preserves an existing attempt, including mistakes. Use a new ID for a fresh attempt. Old stage-1 attempts remain unchanged. In nano, save with **Ctrl+O**, **Enter**; exit with **Ctrl+X** or **F2** if the browser intercepts Ctrl+X. Use Control on a Mac too.

Save changes before leaving. Files in lab-work/ are ignored by Git, not backed up to GitHub. Resume this Codespace; download your work when a separate copy is needed.

Folder usage and free space

The event team has notices, posters and recordings. Predict which folder uses the most storage, then measure it. The .bin files are synthetic exports, not playable media; do not open them in cat or nano.

My prediction and reason:

```
du -sh practice/notices practice/posters practice/recordings
du -sh practice/notices practice/posters practice/recordings > work/guided-usage.txt
```

du estimates allocated storage used by the named files and folders. **-s** gives one summary per path, including its contents; **-h** uses readable units. Here K, M and G increase in steps of 1,024. Compare units as well as numbers.

notices: _____ posters: _____ recordings: _____

Which is larger, 900K or 2.0M? Why?

Read the containing filesystem

```
df -h . > work/filesystem.txt
cat work/filesystem.txt
```

df reports filesystem space. The dot selects the filesystem containing your current folder; it does not add up just that folder. These readings describe the remote Linux environment, not your laptop.

Column	Meaning
Size / Used	Total filesystem capacity / space already used.
Avail / Use%	Available space / percentage used, not free.
Mounted on	Where the filesystem is attached to the directory tree.

```
nano work/observations.txt
```

Fill **guided_largest** with the folder name. Fill the five **filesystem_** fields from your saved df row, keeping units, the percent sign and full mount path. Leave challenge_largest for the next page.

Why does df Used include more than this lab folder?

Make a copy and compare

Predict what will change after copying one recording. The target starts empty. Run this sequence once; when resuming, read the saved reports instead of replacing the before measurement.

My prediction for du and df:

```
ls work/copy-target
du -sh work/copy-target > work/copy-before.txt
cp practice/recordings/recording.bin work/copy-target/
du -sh work/copy-target > work/copy-after.txt
cat work/copy-before.txt work/copy-after.txt
du -sh practice/recordings
du -sh .
df -h .
```

Before: _____ After: _____ Evidence the original remains:

An empty directory may occupy storage. A small copy can increase du while rounded df values look unchanged. Other activity can also change filesystem usage; Used + Avail need not exactly equal Size.

Apply it independently

Measure **challenge/design**, **challenge/notes** and **challenge/interviews** separately. Save their three du summaries as **work/challenge-usage.txt**. Fill **challenge_largest** in observations.txt with the largest folder name.

My command and largest folder, with its displayed size:

The magazine team asks how much more it can save. Run the appropriate command, then record the column and its value.

Command: _____ Column and value: _____

A friend says "du -sh . tells me how much more I can save." Correct the claim.

RAM is not disk space

Disk storage holds saved files. RAM holds code and data used by running programs. Closing a program can release RAM without deleting its saved files. An archive needs disk space; opening it may also require memory.

Read a memory snapshot

```
free -h > work/memory.txt
cat work/memory.txt
df -h .
```

free reports memory; **-h** adds readable units such as Mi and Gi. Read the **Mem:** row for RAM and the **Swap:** row for swap. Swap is storage used to support memory management; it is not extra physical RAM.

Memory column	What it tells you
total	Usable RAM reported by the environment.
used / free	Memory in use / memory currently unused.
buff/cache	Memory used for buffers and caches; some can be reclaimed.
available	An estimate of memory available for new applications without swapping.

A small free value alone does not prove a memory shortage: useful cached data occupies RAM too. Look at available. Swap total may be zero in this environment.

```
nano work/system.txt
```

Fill **memory_total** and **memory_available** from Mem, and **swap_total** from Swap in your saved snapshot. Keep each displayed unit. Leave the remaining fields for the next page.

My Mem total: _____ available: _____ Swap total: _____

Why can available be larger than free?

Container readings can reflect the host kernel view and may differ from the Codespace resource limit. Do not treat free or top as a statement of your account quota. No fixed reading is expected.

Uptime and load averages

```
uptime > work/uptime.txt
cat work/uptime.txt
```

uptime shows the current time, elapsed system uptime, logged-in user count and three load averages. The averages cover the last **1, 5 and 15 minutes**, in that order. System uptime is not the time spent on this exercise.

Load averages reflect runnable work and work in uninterruptible waits, often I/O. They are not CPU percentages. A value needs context, including the number of CPUs and the kind of waiting; it is not a pass/fail score for your Codespace.

Interpret your snapshot

```
nano work/system.txt
```

Fill **uptime_elapsed** with the text after "up" and before the user count; omit the separating comma. Copy the three decimal values to **load_1**, **load_5** and **load_15**. Use your saved output even if a new run differs.

Elapsed uptime: _____

1 minute: _____ 5 minutes: _____ 15 minutes: _____

Which average reflects the shortest interval? What might a higher 1-minute value than 15-minute value suggest?

Apply it independently

For each request, choose a command, run it and record one relevant reading. Explain your choice; the two readings describe different resources.

Request	Command and reading
Can I store another large video file?	
How much RAM may be available to another application?	
Why would a large df Avail value fail to answer the RAM question?	
Is a load average of 2.00 the same as 2% CPU usage? Explain.	

EXPERIMENT 3

Programs, processes and PIDs

A **program** is stored instructions. A **process** is a running instance. Linux assigns each process a numeric **PID**. The same program can have several processes with different PIDs.

```
ps
python3 ../../terminal-lab-3/lab_process.py start
```

ps takes a process snapshot; plain **ps** normally selects processes of your user on this terminal. The supplied helper starts one designated worker for this attempt. It mostly waits, holds about 8 MiB of data and expires after one hour. Leave it running until Experiment 5.

Helper PID: _____ LAB3 token: _____

In every command below, replace **PID** with the positive number printed by your helper. Do not type the word **PID** or use a classmate's number.

```
ps -p PID -o pid,stat,args
ps -p PID -o pid,stat,args > work/process-before.txt
nano work/process-notes.txt
```

-p selects a PID; **-o** chooses columns. **STAT** is the state: **S** means sleeping/waiting, **R** means running or ready; extra letters add detail. **COMMAND** shows the program and its arguments. Fill **pid**, **state** and **command** from the saved row; keep the complete command text.

What in **COMMAND** identifies this as your designated lab worker?

Apply it independently

```
ps -e -o pid,comm
```

-e selects all visible processes; **comm** is the short command name. Find a program name with more than one PID if one is present. Observe only; do not stop it.

Program and PIDs (or "none in this snapshot"):

Why is the program name alone insufficient to identify one process?

Rerunning the helper while its worker is live reuses it. Once stopped or expired, preserve this attempt and use a new lab ID for another process exercise.

EXPERIMENT 4

Watch CPU and memory with top

top refreshes its display as processes run. The summary describes the system; each process row describes one process. Unlike a saved `ps` snapshot, the live screen changes.

```
top
```

Watch two refreshes. Press **Shift+P** to sort by CPU, then **Shift+M** to sort by memory. Identify the PID and COMMAND of a visible row. Press **q** to return to Bash. These keys act inside `top`.

A row I observed (PID and COMMAND):

```
python3 ../../terminal-lab-3/lab_process.py status
top -p PID
```

Replace PID with your worker's current number. Watch at least two refreshes. The worker should usually be idle; 0.0 CPU or rounded 0.0 memory is a valid observation.

Field	Meaning
%CPU	CPU usage over the sampling interval; not a load average.
RES	Resident memory held in physical RAM; read displayed units.
%MEM	Resident memory as a share of reported physical memory.
VIRT	Virtual address space; not the amount of physical RAM used.

Record two %CPU readings, RES and %MEM in the spaces below. Press `q`, then enter them in **work/process-notes.txt** as **top_cpu_first**, **top_cpu_later**, **top_res** and **top_mem_percent**. Use numeric CPU/%MEM values without a percent sign.

CPU first: _____ later: _____ RES: _____ (unit: _____) %MEM: _____

```
top -b -n 1 -p PID > work/top.txt
```

-b produces text output; **-n 1** limits it to one snapshot. Keep the interactive observations too: a batch report cannot show that you used the live controls.

Apply: choose `top` or `ps` to watch a changing process list. Why?

Does a sleeping process with 0.0 CPU use no memory? Explain.

EXPERIMENT 5

Stop only the lab worker

kill sends a signal to a process. With no option it requests termination using SIGTERM, giving the program a chance to clean up. It does not delete the program file. For this exercise, stop only the worker created for your attempt.

Confirm the identity immediately before stopping

```
python3 ../../terminal-lab-3/lab_process.py status
ps -p PID -o pid,stat,args > work/process-confirm.txt
cat work/process-confirm.txt
```

Continue only if the helper says **RUNNING** and the ps row contains **lab_process.py worker** and your exact **LAB3 token** from page 6. Replace PID with that same current number. If anything differs, stop and ask for help. Never guess a PID, use 0 or a negative number, or signal another visible process.

Confirmed PID: _____ Matching token: _____

```
kill PID
ps -p PID -o pid,stat,args > work/process-after.txt
cat work/process-after.txt
python3 ../../terminal-lab-3/lab_process.py status
```

Expect the ps header with no data row and helper status **STOPPED**. When ps finds no matching process it can return a nonzero exit status; the header is still useful evidence. If a row briefly remains, wait a moment and repeat only the ps check. Do not send more signals blindly.

What evidence shows that the designated worker stopped?

Apply it independently

Inspect the recorded PID again with ps. Explain why you must not send kill again just because you remember that number: Linux can later reuse a PID for another process.

My command and explanation:

Why does an empty terminal response from kill alone not prove completion?

Do not use kill -9, broad name-based stopping or top's kill action. If the worker expired before this step, preserve this attempt and use a new ID to repeat Experiments 3-5.

Package files with zip

An **archive** stores several files and their paths in one file. A ZIP archive can also compress file data. Packaging is useful only if the expected files can be listed, recovered and compared with the originals.

```
ls practice/dispatch
ls practice/dispatch/lists
cat practice/dispatch/notice.txt
```

Predict: will archiving remove the originals?

```
zip -r work/dispatch.zip practice/dispatch
unzip -l work/dispatch.zip > work/zip-list.txt
cat work/zip-list.txt
```

zip -r includes the folder recursively, including nested files. The archive name comes before the source folder. **unzip -l** lists contents without extracting. Paths are relative to the folder where you ran zip.

Which listed path proves that the nested room list is included?

Extract into a separate folder

```
unzip work/dispatch.zip -d work/unzipped
diff -r practice/dispatch work/unzipped/practice/dispatch > work/zip-diff.txt
cat work/zip-diff.txt
```

-d selects the extraction destination; unzip creates it if needed. **diff -r** compares corresponding files through the directory trees. An empty comparison report means no differences were found. Inspect any printed difference before continuing.

My extraction path and comparison result:

Why is seeing an archive filename in ls insufficient verification?

Create each named archive once. zip updates an existing archive and can retain old entries. Re-extraction may ask about overwriting; do not approve blindly. Recovery guidance is on page 14.

Send a magazine handover

The magazine team has a new handover in **challenge/dispatch/**. Complete this without copying the guided command block unchanged. Keep the nested team folder and both text files.

```
cat challenge/dispatch/brief.txt
ls challenge/dispatch
ls challenge/dispatch/team
```

Build, inspect and recover

1. Create **work/magazine.zip** from the complete challenge/dispatch folder.

My packaging command:

2. Save its listing as **work/magazine-list.txt**. Inspect the paths before extracting.

My listing command and the nested path I checked:

3. Extract into **work/magazine-out/**. Work out where the original folder path will appear inside this destination.

My extraction command:

4. Compare the recovered dispatch folder with challenge/dispatch. Save the recursive diff output as **work/magazine-diff.txt**.

My comparison command and result:

Understanding check

A ZIP lists brief.txt but no team/credits.txt. What likely went wrong, and how would you check before sending it?

Does a successful extraction prove that the source and recovered files match? Why?

Archiving versus compression

tar collects files and paths into an archive. Plain tar does not compress their data. Adding gzip compression can reduce the archive size; how much depends on the data. Changing a filename extension does not change its contents.

The supplied **practice/compress/** folder has repeated text and deterministic binary noise of similar size. Predict which would compress more. Do not display noise.bin as text.

My prediction and reason:

```
ls -l practice/compress
tar -cf work/bundle.tar practice/compress
tar -tf work/bundle.tar > work/tar-list.txt
cat work/tar-list.txt
```

-c creates; **-t** lists; **-f** takes the archive filename immediately after the options. The source folder follows. A directory is included recursively.

```
tar -czf work/bundle.tar.gz practice/compress
tar -tzf work/bundle.tar.gz > work/targz-list.txt
cat work/targz-list.txt
du -h work/bundle.tar work/bundle.tar.gz > work/archive-sizes.txt
cat work/archive-sizes.txt
```

-z adds gzip compression. Both listings should describe the same source files. **du** measures allocated disk usage; **ls -l** shows logical file bytes. Tiny archives may occupy equal disk blocks even if byte sizes differ.

tar size: _____ tar.gz size: _____ Compare with units:

If both contain the same files, why can their sizes differ?

A combined archive cannot show which individual sample compressed best. On the next page, measure each sample separately to test your prediction.

Recover and test your prediction

```
mkdir -p work/tar-out
tar -xzf work/bundle.tar.gz -C work/tar-out
diff -r practice/compress work/tar-out/practice/compress > work/tar-diff.txt
cat work/tar-diff.txt
```

-x extracts; **-C** chooses an existing destination folder. **mkdir -p** creates it if missing. Extract these supplied archives only into **work/**, then compare the recovered files with the originals.

My comparison result and what it means:

Test the two samples separately

```
tar -czf work/text.tar.gz practice/compress/repeated.txt
tar -czf work/noise.tar.gz practice/compress/noise.bin
ls -l practice/compress/repeated.txt practice/compress/noise.bin
ls -l work/text.tar.gz work/noise.tar.gz
```

Text bytes before/after: _____ / _____

Noise bytes before/after: _____ / _____

Did the result support your prediction? Explain any archive overhead.

Apply it independently

Create **work/magazine.tar.gz** from **challenge/dispatch**. List it to confirm that both files, including the nested credits file, are included.

My create and list commands:

A friend renames **bundle.tar** to **bundle.tar.gz**. Has compression occurred? How would you distinguish a name from an operation?

An archive does not replace the originals or guarantee a backup. Keep a separately stored, verified copy when you need recovery beyond this Codespace.

Review your evidence

```
python3 ../../terminal-lab-3/check.py
```

Run the checker from your lab root at any point. **PASS** means a saved result meets that check. **CHECK** names something to inspect. Future experiments remain unfinished until you do them. The full book has **35 checks**; the checker never repairs, uploads or submits work.

Experiment	Evidence in work/
1	usage reports, filesystem.txt, observations.txt, before/after copy and recording copy
2	memory.txt, uptime.txt, system.txt
3-5	process-before/confirm/after.txt, process-notes.txt, top.txt and helper records
6	both ZIPs, listings, recovered folders and diff reports
7	tar and tar.gz files, listings, recovered folder, diff and size reports

File checks cannot grade explanation quality or prove that you used interactive tools independently. Show your instructor your live top controls and discuss the choices below.

1. Which commands answer "largest folder", "available storage" and "available RAM"? Why are these different questions?

2. What would you verify before stopping a process, even if you wrote its PID down earlier?

3. What evidence would you provide with an archive to show that its nested files can be recovered correctly?

4. Does a larger load average necessarily mean a higher CPU percentage? Explain what else you need to know.

Save notes and editor buffers. Keep this Codespace for review. Your instructor may ask you to download work/ separately; the checker is feedback, not a submission system.

When a result surprises you

Check location and saved evidence

Use **pwd** before relative paths. Return to the lab root recorded on page 1. Read the saved report with **cat**; do not fill observation fields from a newer snapshot. The **>** symbol replaces its destination, so point it only at the intended work/ file.

Storage and memory

Compare units before numbers. **du** summarizes named folders; **df** describes their containing filesystem. In free, find the Mem row and available column. In uptime, the final three values are ordered from shortest to longest interval. Readings may change without an action from you.

Processes

Use the helper's actual PID wherever the book says PID. Match the complete LAB3 token in the **ps** command line. An S state or 0.0 CPU can be normal. **top** may show a short program name; the **ps** arguments are the stronger identity check. Press **q** to leave **top** before entering shell commands.

If the helper says STOPPED before Experiment 5, do not reuse that PID. It may have expired after an hour or the Codespace may have stopped. Preserve your files and use a new lab ID for a fresh process attempt. The helper will not silently replace a stopped worker.

Archives

An archive stores paths relative to where you created it. Listing first tells you which path to expect after extraction. ZIP uses **-d** for the destination; tar uses **-C** and needs that folder to exist. Check the archive name after **-f** carefully.

If you must redo a ZIP, first move the old archive and extraction folder to unused backup names inside work/ using familiar **mv**. Then recreate and extract into the now-unused names from the book. Do not update a suspect archive repeatedly or overwrite originals. Ask for help if unsure.

Read checker feedback

Fix the named result, save, and run the checker again. A missing report differs from an incorrect explanation. If original files or before-copy evidence changed, preserve the attempt and ask for a new ID. Setup reruns do not reset or repair anything.

Use the manual when needed

Use **man COMMAND** for the Linux command installed here; **q** exits the manual. Read options in context before trying them. The lab requires only the commands shown in this book.

Ten new commands

Command / example	Purpose
<code>du -sh FOLDER</code>	One readable allocated-storage summary.
<code>df -h .</code>	Capacity and space of the containing filesystem.
<code>free -h</code>	Readable RAM and swap snapshot.
<code>uptime</code>	Elapsed uptime; 1, 5 and 15 minute load averages.
<code>ps -p PID -o pid,stat,args</code>	Snapshot of one PID, its state and command line.
<code>top / top -p PID</code>	Live process display / one selected PID.
<code>kill PID</code>	Request termination of the confirmed lab worker.
<code>zip -r work/name.zip FOLDER</code>	Package a folder and its nested contents.
<code>unzip -l work/name.zip</code>	List a ZIP archive.
<code>unzip work/name.zip -d work/out</code>	Extract ZIP into the named destination.
<code>tar -cf work/name.tar FOLDER</code>	Create an uncompressed tar archive.
<code>tar -czf work/name.tar.gz FOLDER</code>	Create a gzip-compressed tar archive.
<code>tar -tf work/name.tar</code>	List tar contents; use <code>-tzf</code> for gzip.
<code>tar -xzf work/name.tar.gz -C work/out</code>	Extract gzip tar into an existing folder.

Familiar tools and keys

pwd / cd / ls: location, navigation, listing. **cat**: read text. **cp / mv**: copy or move a named file. **mkdir -p**: create a destination folder. **diff -r A B**: compare folder trees. **COMMAND > work/file**: save output, replacing that file. **nano**: edit notes.

nano: Ctrl+O, Enter saves; Ctrl+X or F2 exits. **top**: Shift+P sorts CPU, Shift+M sorts memory, q exits. **top -b -n 1 -p PID** saves a single batch snapshot when redirected.

References: [GNU file space usage](#); [free](#), [uptime](#), [ps](#), [top](#); installed **man zip**, **man unzip**, **man tar** and Bash **help kill**.